

IF2211 ALGORITHM STRATEGY

Pattern Matching in Chromium Browser Extension for Online Gambling Detection

May 29, 2026



Made Branenda Jordhy - 13524026

Stefani Angeline Oroh - 13524064

Michael James Liman - 13524106

School of Electrical Engineering and Informatics
Institut Teknologi Bandung
Semester II 2025/2026

Contents

1 Overview	3
2 Theoretical Background	4
2.1 Algoritma Knuth-Morris-Pratt (KMP)	4
2.2 Algoritma Boyer-Moore (BM)	5
2.3 Algoritma Aho-Corasick (Bonus)	6
2.4 Algoritma Rabin-Karp (Bonus)	7
2.5 Weighted Levenshtein Distance	7
2.6 Regular Expression	8
2.7 Chromium Browser Extension	8
3 Problem Analysis & Solution Design	10
3.1 Langkah-langkah Pemecahan Masalah	10
3.2 Pipeline Pencocokan Tiga Tahap	11
3.3 Pemetaan Masalah ke Elemen KMP dan Boyer-Moore	12
3.4 Fitur Fungsional dan Arsitektur	12
3.5 Contoh Ilustrasi Kasus	13
4 Implementasi dan Pengujian	15
4.1 Spesifikasi Teknis Program	15
4.2 Tata Cara Penggunaan Ekstensi	16
4.3 Hasil Pengujian	17
4.3.1 Kasus 1: Hasil Pencarian Google dengan Kata Kunci Judol	17
4.3.2 Kasus 2: Komentar Spam Judol pada Halaman Video	18
4.3.3 Kasus 3: Halaman Situs Judol Langsung	18
4.3.4 Kasus 4: Halaman dengan Penyamaran Karakter Visual	19
4.3.5 Kasus 5: Iklan Judol Berbentuk Gambar (OCR)	20
4.4 Analisis Perbandingan Performa	20
5 Kesimpulan dan Saran	22
Appendix	24

1 Overview

Proyek ini kami kerjakan untuk memenuhi Tugas Besar 3 mata kuliah IF2211 Strategi Algoritma. Topiknya sudah ditentukan, yaitu membuat sebuah *Chromium browser extension* yang bisa mendeteksi konten judi online di halaman web, sekaligus jadi tempat menerapkan algoritma pencocokan string yang dipelajari di kelas.

Topik judol sendiri sebetulnya relevan jika dilihat dari kondisi sekarang. Iklannya ada di mana-mana, mulai dari hasil pencarian Google, kolom komentar YouTube, sampai sisipan iklan di situs yang sebenarnya tidak ada hubungannya sama sekali dengan perjudian. Nama-nama yang dipakai pun mirip semua, biasanya berpola <kata><angka> seperti GACOR99, MAXWIN88, atau MADU308, supaya gampang diingat dan susah disaring otomatis. Tidak jarang pelakunya juga menyamarkan tulisan dengan karakter yang bentuknya mirip, misalnya huruf O jadi angka 0, A jadi 4, atau I jadi 1, sehingga filter yang cuma membandingkan string mentah sering kebobolan.

Ekstensi yang kami bangun untuk tugas ini kami beri nama **Judol Detector**. Begitu halaman selesai dimuat, ekstensi akan menelusuri DOM, mencocokkan teks dengan daftar kata kunci, lalu menandai elemen yang mengandung kemunculan kata kunci tersebut. Pengguna bisa mengarahkan kursor ke teks yang ditandai untuk melihat detailnya, atau membuka panel popup ekstensi untuk melihat ringkasan statistik secara keseluruhan. Pencocokan kata kunci dijalankan dengan beberapa algoritma sekaligus, yaitu Knuth-Morris-Pratt dan Boyer-Moore untuk tahap pencocokan eksak, RegEx untuk menangkap pola <kata><angka> yang belum tentu ada di daftar kata kunci, lalu Weighted Levenshtein Distance untuk tahap fuzzy yang menangani kasus penyamaran karakter. Seluruh algoritma string matching dibuat *from scratch*, tanpa bantuan `String.includes()`, `String.indexOf()`, atau pustaka pencocokan string apapun. Tiap algoritma juga menghitung jumlah perbandingan karakter (*comparison counting*) supaya performanya bisa dibandingkan secara objektif.

2 Theoretical Background

2.1 Algoritma Knuth-Morris-Pratt (KMP)

KMP pada dasarnya merupakan satu observasi sederhana pada matching brute-force. Jika pencarian gagal di tengah jalan, sebetulnya kita sudah punya cukup informasi dari karakter yang sudah dibandingkan untuk tahu sejauh mana pola harus digeser. Brute-force tidak peduli soal itu dan mengembalikan pointer teks ke posisi sebelumnya, jadi banyak perbandingan yang sudah dilakukan terbuang sia-sia. KMP memanfaatkan informasi tadi supaya pointer teks bisa terus maju. Hasilnya, kompleksitas pencariannya $O(n + m)$ untuk teks berukuran n dan pola berukuran m .

Kunci algoritma ini adalah *border function*, yaitu tabel yang menyimpan panjang *proper prefix* terpanjang pada pola yang sekaligus juga merupakan suffix-nya, sampai posisi tertentu. Tabel ini dihitung sekali di awal dengan kompleksitas $O(m)$. Kalau mismatch terjadi di indeks i pada pola, pointer pola dipindahkan ke posisi $b[i - 1]$, sementara pointer teks tetap di tempatnya. Contoh nilai border function untuk pola **ababaca** bisa dilihat di Tabel 1.

Table 1: Border function untuk pola **ababaca**.

i	0	1	2	3	4	5	6
$P[i]$	a	b	a	b	a	c	a
$b[i]$	0	0	1	2	3	0	1

Kompleksitas pencariannya sebetulnya linier baik di kasus terbaik maupun terburuk, cara mencapainya saja yang berbeda [2]. Misalkan pola berukuran m dicari pada teks berukuran n . Kasus paling sederhana terjadi saat pola langsung ditemukan di awal teks dengan biaya $O(m)$.

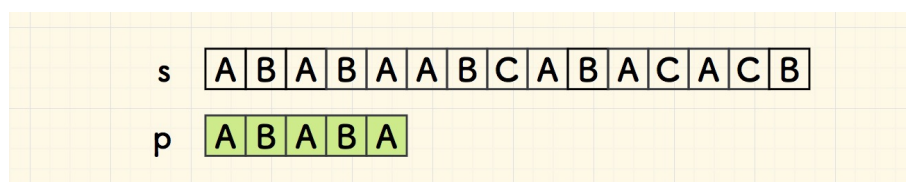


Figure 1: Kasus terbaik trivial: pola langsung ditemukan di awal teks (source: [stackoverflow](#)).

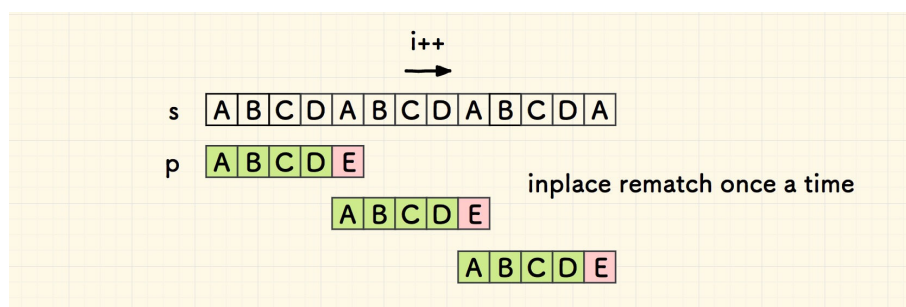


Figure 2: Kasus terbaik saat pola tidak ada di teks: tiap mismatch jatuh di karakter pertama pola (source: [stackoverflow](#)).

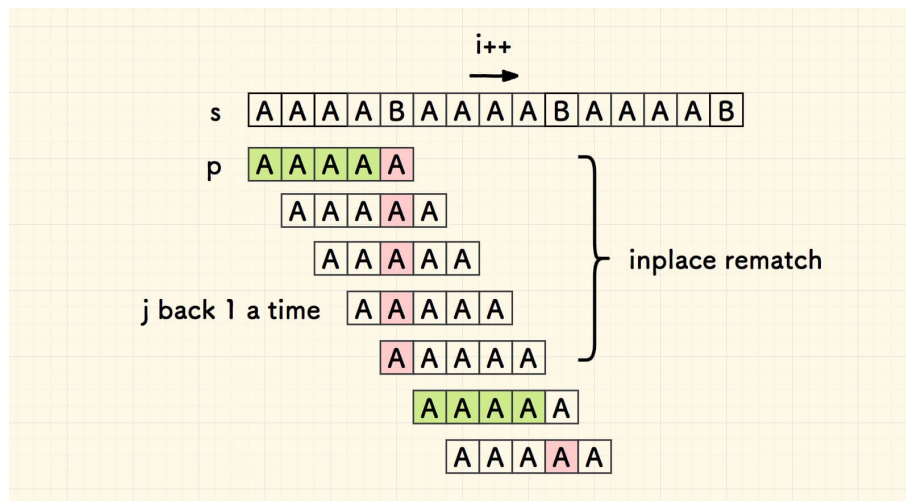


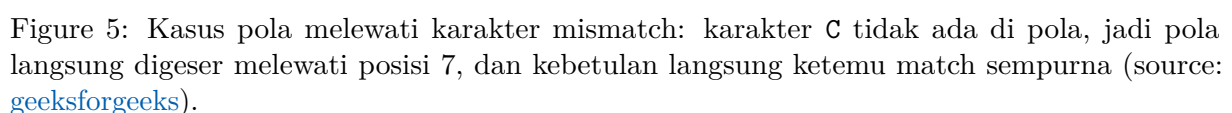
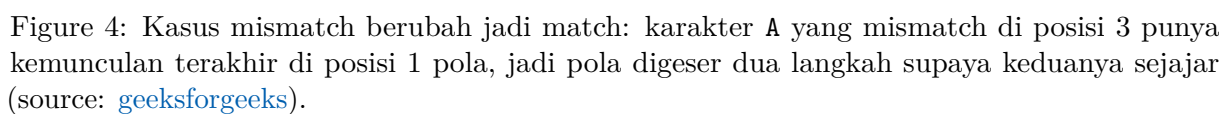
Figure 3: Kasus terburuk: mismatch baru terjadi setelah hampir seluruh karakter pola dibandingkan (source: [stackoverflow](#)).

Di implementasi kami, perbandingan saat membangun border function juga ikut dihitung sebagai *comparison count*, bukan cuma yang terjadi saat pencarian. Angka inilah yang nantinya dipakai untuk membandingkan performa antar algoritma di Bab 4.

2.2 Algoritma Boyer-Moore (BM)

Ciri utama Boyer-Moore adalah arah perbandingannya. Pointer pola tetap maju dari kiri ke kanan terhadap teks, tapi karakter pola dibandingkan dari kanan ke kiri. Cara ini memungkinkan pola melompat banyak karakter sekaligus saat mismatch, dan praktiknya sering lebih cepat dari KMP terutama untuk pola yang panjang. Boyer-Moore standar punya dua heuristik, *bad character* dan *good suffix*. Yang kami pakai di sini adalah *bad character* via **last occurrence table**, yaitu tabel posisi kemunculan terakhir tiap karakter pada pola.

Saat mismatch terjadi, ada dua kemungkinan yang ditangani heuristik ini [3]. Pertama, kalau karakter mismatch masih ada di pola, pola digeser supaya kemunculan terakhir karakter tersebut sejajar dengan posisi mismatch di teks (Figure 4). Kedua, kalau karakter mismatch tidak ada di pola sama sekali, pola langsung digeser melewati posisi mismatch tersebut (Figure 5).



2.3 Algoritma Aho-Corasick (Bonus)

Failure link di sini mirip dengan border function pada KMP. Kalau mismatch terjadi di sebuah simpul, automaton tidak balik ke akar, tapi melompat ke simpul terdalam yang merepresentasikan *proper suffix* terpanjang dari string yang sudah dicocokkan sejauh ini [4]. Contoh struktur automatonnya dapat dilihat pada Figure 6.

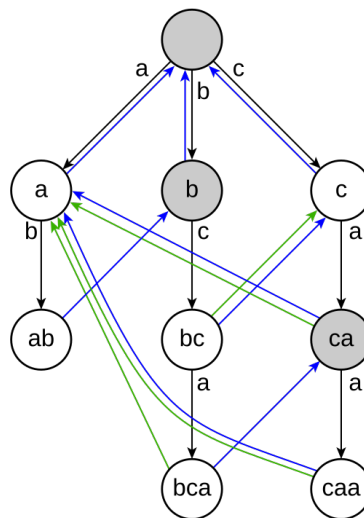


Figure 6: Contoh struktur automaton Aho-Corasick. Panah hitam adalah *goto transition* dan panah biru adalah *failure link* yang menunjuk ke simpul terdalam yang merepresentasikan *proper suffix* terpanjang dari string yang sudah dicocokkan (sumber: [wikipedia](https://en.wikipedia.org/wiki/Aho%E2%80%93Corasick_algorithm)).

Untuk Judol Detector, daftar kata kunci judul bisa cukup panjang. Dengan KMP atau BM, teks harus diulang sebanyak jumlah kata kunci. Aho-Corasick cukup sekali jalan.

2.4 Algoritma Rabin-Karp (Bonus)

Rabin-Karp memakai pendekatan berbasis *hashing*. Daripada membandingkan karakter satu per satu untuk tiap posisi pada teks, algoritma ini menghitung nilai hash dari setiap *window* berukuran m di teks, lalu membandingkannya dengan hash pola. Cuma kalau hash-nya cocok, baru dilakukan pengecekan karakter penuh untuk memastikan bukan tabrakan hash (*spurious hit*).

Yang membuat Rabin-Karp tetap efisien adalah **rolling hash**, biasanya Rabin fingerprint, yang memungkinkan perhitungan hash window berikutnya cuma $O(1)$ dari hash window sebelumnya. Kompleksitas rata-ratanya $O(n + m)$, walau worst-case-nya tetap $O(nm)$ kalau hash sering bertabrakan. Rabin-Karp juga cocok untuk pencocokan multi-pola karena hash banyak pola bisa disimpan di tabel dan diuji bersamaan terhadap *rolling hash* teks.

2.5 Weighted Levenshtein Distance

Levenshtein distance adalah jumlah operasi minimum (*insert*, *delete*, *substitute*) untuk mengubah satu string jadi string lain. Versi standarnya memberi bobot yang sama untuk semua operasi, biasanya 1, dan dihitung pakai *dynamic programming* dalam waktu $O(nm)$.

Pada varian *Weighted Levenshtein*, bobot tiap operasi bisa disetel berbeda-beda. Di kasus judul, substitusi antar karakter yang mirip secara visual (O dengan 0, I dengan 1, A dengan 4) diberi bobot lebih kecil daripada substitusi karakter yang benar-benar berbeda. Jadi penyamaran semacam HOKI88 versus H0KI88 tetap dianggap mirip, sementara substitusi acak akan dihukum penuh. Tabel 2 mengilustrasikan perhitungannya untuk pasangan OKI dan OKI.

Table 2: Matriks DP Weighted Levenshtein untuk mengubah OKI menjadi OKI, dengan bobot substitusi $O \leftrightarrow 0$ sebesar 0.3 dan operasi lain bernilai 1. Sel pojok kanan bawah (cetak tebal) adalah jaraknya, yaitu 0.3. Dengan bobot standar (semua substitusi bernilai 1), nilainya akan jadi 1, sehingga ambang batas kemiripan biasanya tidak tercapai.

	ε	O	K	I
ε	0	1	2	3
O	1	0.3	1.3	2.3
K	2	1.3	0.3	1.3
I	3	2.3	1.3	0.3

Untuk membatasi sampai mana dua string masih dianggap cocok, dipakai ambang batas (*threshold*) pada skor kemiripan ($1 - \text{normalized distance}$). Perhitungan juga dipecah per token (split berdasarkan spasi) supaya teks panjang tidak salah cocok karena *distance* menumpuk dari banyak kata sekaligus.

2.6 Regular Expression

Regular expression (Regex) adalah bahasa untuk mendefinisikan pola tekstual secara formal. Mesin Regex modern, termasuk yang ada di JavaScript, biasanya berbasis NFA dengan *backtracking*. Di sini, Regex kami pakai untuk menangkap pola `<kata><angka>` yang sering jadi ciri khas nama situs judol, yaitu kombinasi minimal dua huruf diikuti minimal dua angka, dibatasi oleh *word boundary*.

Beda dari KMP atau BM yang butuh daftar kata kunci yang sudah diketahui, Regex berperan sebagai *discovery layer*. Kalau ada nama situs baru yang belum masuk daftar kata kunci tapi tetap mengikuti pola tersebut, ekstensi tetap bisa mendeteksinya.

2.7 Chromium Browser Extension

Chromium browser extension adalah perangkat lunak kecil yang dipasang di browser berbasis Chromium (Chrome, Edge, Brave, dan sejenisnya), yang bisa menambah atau memodifikasi fungsi browser saat menjelajah web. Sejak **Manifest V3**, sebuah ekstensi umumnya terdiri dari beberapa komponen utama.

- **Manifest** (`manifest.json`), berkas konfigurasi yang mendeklarasikan nama, versi, izin (*permissions*), dan daftar komponen ekstensi.
- **Content script**, skrip yang disuntikkan langsung ke halaman web dan bisa berinteraksi dengan DOM. Inilah komponen yang menjalankan pemindaian teks pada Judol Detector.
- **Popup**, halaman kecil yang muncul saat ikon ekstensi diklik, biasanya berisi UI untuk menampilkan ringkasan informasi dan tombol-tombol kontrol.
- **Service worker** (background), skrip latar yang menangani *event* global dan menyimpan *state* ekstensi. Pada Manifest V3, service worker hanya hidup saat dibutuhkan, berbeda dengan background page Manifest V2 yang persisten.

Ketiga komponen ini berkomunikasi lewat API `chrome.runtime`, biasanya pakai pesan (`sendMessage` dan `onMessage`) atau lewat `chrome.storage` kalau perlu berbagi *state*. Arsitektur Judol Detector secara umum tergambar pada Figure 7.

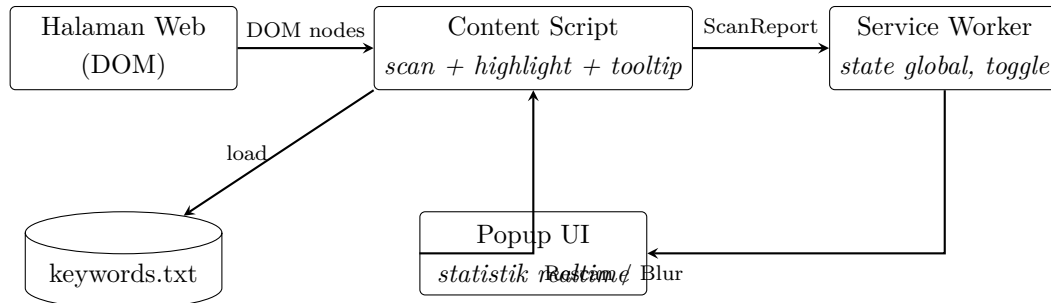


Figure 7: Arsitektur Judol Detector. *Content script* adalah pemain utama yang menyentuh DOM dan menjalankan algoritma pencocokan, sementara popup dan *service worker* berperan sebagai pengontrol dan penampung *state*.

Pada Judol Detector, *content script* bertugas menelusuri DOM dan menjalankan algoritma pencocokan, popup menampilkan statistik *realtime* hasil pemindaian beserta kontrol *blur toggle* dan *rescan*, sedangkan *service worker* memelihara *state* global seperti status *blur* antar tab.

3 Problem Analysis & Solution Design

3.1 Langkah-langkah Pemecahan Masalah

Intinya adalah memutuskan, dengan cepat dan tepat, apakah teks yang muncul di sebuah halaman web mengandung konten judul. Untuk hal itu, kami pecah alurnya menjadi lima tahap yang berjalan berurutan, dari saat page selesai dimuat sampai informasinya sampai ke pengguna. Kelima tahap tersebut diringkas sebagai berikut.

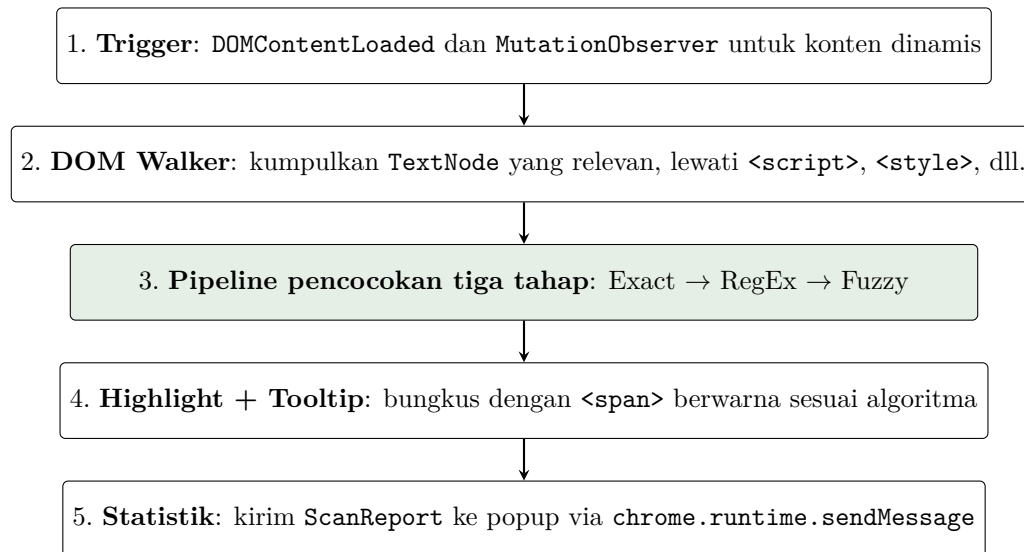


Figure 8: Lima tahap pemecahan masalah Judol Detector, dari *trigger* sampai laporan akhir di popup. Tahap ketiga diperdetail pada Figure 9.

Tahap pertama adalah *trigger*. Begitu DOM selesai dimuat, *content script* dipanggil dan memulai pemindaian. Konten yang muncul belakangan seperti hasil *infinite scroll* atau *lazy load* ditangani oleh *MutationObserver* yang otomatis memicu pemindaian ulang pada *subtree* baru.

Tahap kedua adalah penelusuran DOM. DOM walker menelusuri seluruh *TextNode* di halaman, mengumpulkan teksnya, lalu melewati elemen yang tidak relevan seperti *<script>*, *<style>*, *<noscript>*, dan elemen yang sudah pernah ditandai. Hasilnya adalah daftar teks yang siap dicocokkan, lengkap dengan referensi ke node aslinya supaya highlighter tahu di mana harus membungkus ** nantinya.

Tahap ketiga adalah pipeline pencocokan tiga lapis yang jadi inti deteksi.

Tahap keempat adalah pemasangan highlight dan tooltip. Tiap kemunculan yang lolos pipeline dibungkus dengan ** berwarna berbeda per algoritma penemu. Saat pengguna meng-hover *span* tersebut, tooltip kustom muncul dan menampilkan kata kunci, algoritma penangkap, jumlah kemunculan, serta waktu eksekusi.

Tahap kelima adalah laporan ke popup. Setelah pemindaian selesai, *content script* mengirim *ScanReport* berisi seluruh statistik melalui *chrome.runtime.sendMessage*. Popup menampilkan total kata kunci yang ditemukan, perbandingan jumlah *match* dan jumlah perbandingan per algoritma, serta grafik perbandingan waktu eksekusi.

3.2 Pipeline Pencocokan Tiga Tahap

Pipeline pencocokan tiga tahap ini yang melakukan kerja deteksi yang sebenarnya. Tiap tahap melengkapi tahap sebelumnya seperti *exact* menangkap kata kunci yang sudah dikenal, RegEx menambah penangkapan nama berpola `<kata><angka>` yang belum dikenal, dan *fuzzy* menangkap kata kunci yang mengalami penyamaran karakter.

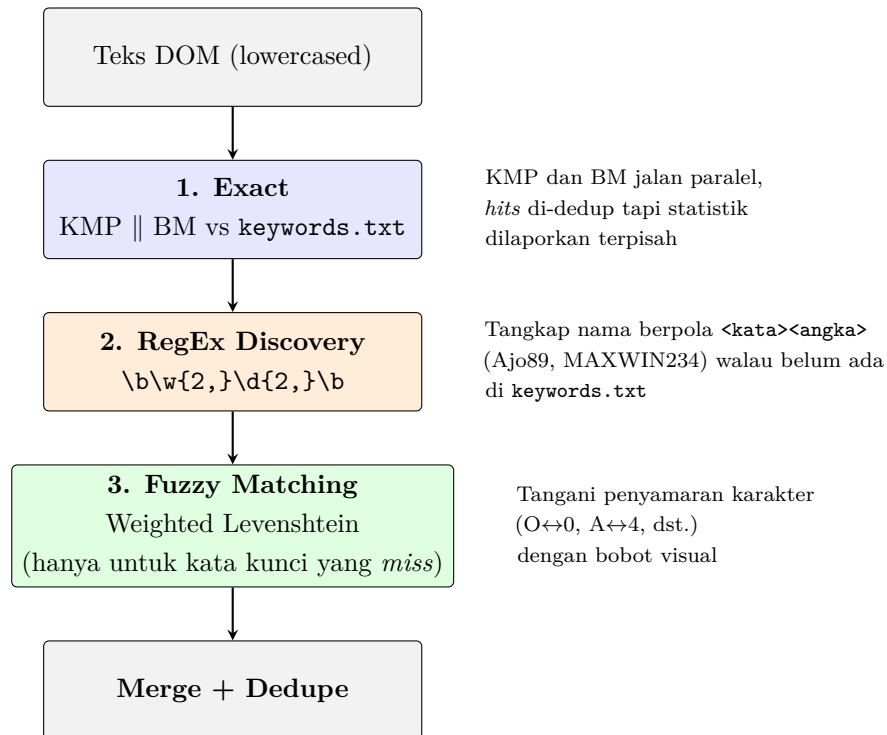


Figure 9: Detail pipeline tiga tahap. *Exact* menangani kata kunci dasar, RegEx menambah penemuan nama baru, dan *fuzzy* menjaring penyamaran karakter. Hasil semua tahap digabung dan di-dedup sebelum dilanjutkan ke highlighter.

Pada tahap *exact*, KMP dan Boyer-Moore dijalankan paralel terhadap seluruh kata kunci di `keywords.txt`. Karena keduanya seharusnya menemukan kemunculan yang sama untuk pola yang sama, hasilnya di-dedup di orchestrator. Walaupun begitu, statistik per algoritma (jumlah match, comparison count, waktu eksekusi) tetap di-report terpisah supaya bisa dibandingkan secara objektif nantinya.

Pada tahap RegEx, satu pola `\b\w{2,}\d{2,}\b` menangkap kombinasi minimal dua huruf diikuti minimal dua angka, dibatasi *word boundary*. Tahap ini tidak menyaring berdasarkan kata kunci, melainkan menemukan nama-nama berpola `<kata><angka>` secara umum, jadi nama baru seperti misal Ajo89 atau MAXWIN234 tetap tertangkap walau belum pernah didaftarkan.

Pada tahap *fuzzy*, Weighted Levenshtein hanya dipanggil untuk kata kunci yang tidak ketemu di tahap eksak (*miss list*). Tiap token di teks dicocokkan dengan kata kunci yang *miss*, dan kemunculan ditandai `isFuzzy = true` jika skor kemiripannya di atas *threshold* 0.7.

3.3 Pemetaan Masalah ke Elemen KMP dan Boyer-Moore

Sebelum pipeline jalan, persoalan deteksi judul harus dipetakan ke elemen-elemen klasik string matching. Untuk KMP dan Boyer-Moore, pemetaannya diringkas pada Tabel 3. Karena tipe input dan output kedua algoritma identik, *orchestrator* bisa langsung memanggil keduanya untuk satu pola dan membandingkan statistiknya.

Table 3: Pemetaan masalah deteksi judul ke elemen-elemen algoritma KMP dan Boyer-Moore.

Elemen	KMP	Boyer-Moore
Teks input T	Teks dari satu <code>TextNode</code> , sudah dinormalisasi ke lowercase	Sama
Pola P	Satu kata kunci dari <code>keywords.txt</code> (lowercased)	Sama
<i>Preprocessing</i>	Border function $b[0..m-1]$, biaya $O(m)$	Last occurrence table $\lambda[c]$ untuk tiap karakter c pada Σ
Arah perbandingan	Kiri ke kanan	Kanan ke kiri
Aksi saat mismatch di indeks i	Pointer pola pindah ke $b[i-1]$, pointer teks tetap	Geseran = $\max(1, i - \lambda[c])$
Aksi saat match penuh	Geser satu langkah (<i>overlap</i> diizinkan)	Sama
Comparison count	Akumulatif sejak awal pemanggilan, termasuk saat membangun tabel	Sama
Output	<code>MatchResult[]</code> berisi posisi, keyword, dan <code>comparisonCount</code>	Sama, dengan label algoritma BM

Yang paling penting, kedua algoritma dipanggil untuk *tiap* kata kunci secara terpisah. Untuk daftar berisi K kata kunci, KMP dan BM masing-masing dipanggil K kali, dengan *counter* di-reset di awal tiap pemanggilan. Saat sebuah kemunculan ditemukan, pointer pola hanya digeser satu langkah ke kanan supaya kemunculan yang *overlapping* (misalnya **ababab** terhadap pola **abab**) tetap tertangkap.

3.4 Fitur Fungsional dan Arsitektur

Fitur fungsional Judol Detector dibangun di sekitar pipeline pencocokan tadi.

- Pemindaian otomatis saat halaman selesai dimuat, dengan pemicu ulang untuk konten dinamis.
- Highlight visual pada elemen yang mengandung kata kunci, warna berbeda per algoritma penemu.
- Tooltip kustom saat *hover*: menampilkan keyword, algoritma, jumlah kemunculan, dan waktu eksekusi.

- Popup statistik *realtime*: total kata kunci, perbandingan match dan comparison count per algoritma, dan grafik perbandingan waktu eksekusi.
- Tombol *rescan* dan *toggle blur* di popup.
- Sensor visual otomatis (*blur*) untuk elemen yang teridentifikasi, dengan toggle on/off.
- OCR pada `<img>` via Tesseract.js untuk menangkap kata kunci yang disembunyikan di gambar.

Secara arsitektur, fitur-fitur tersebut terbagi ke dalam tiga komponen utama Manifest V3 seperti yang sudah dibahas di Bab 2. Figure 10 memperlihatkan rincian internalnya, terutama pada *content script* yang menjadi pusat aktivitas pemindaian.

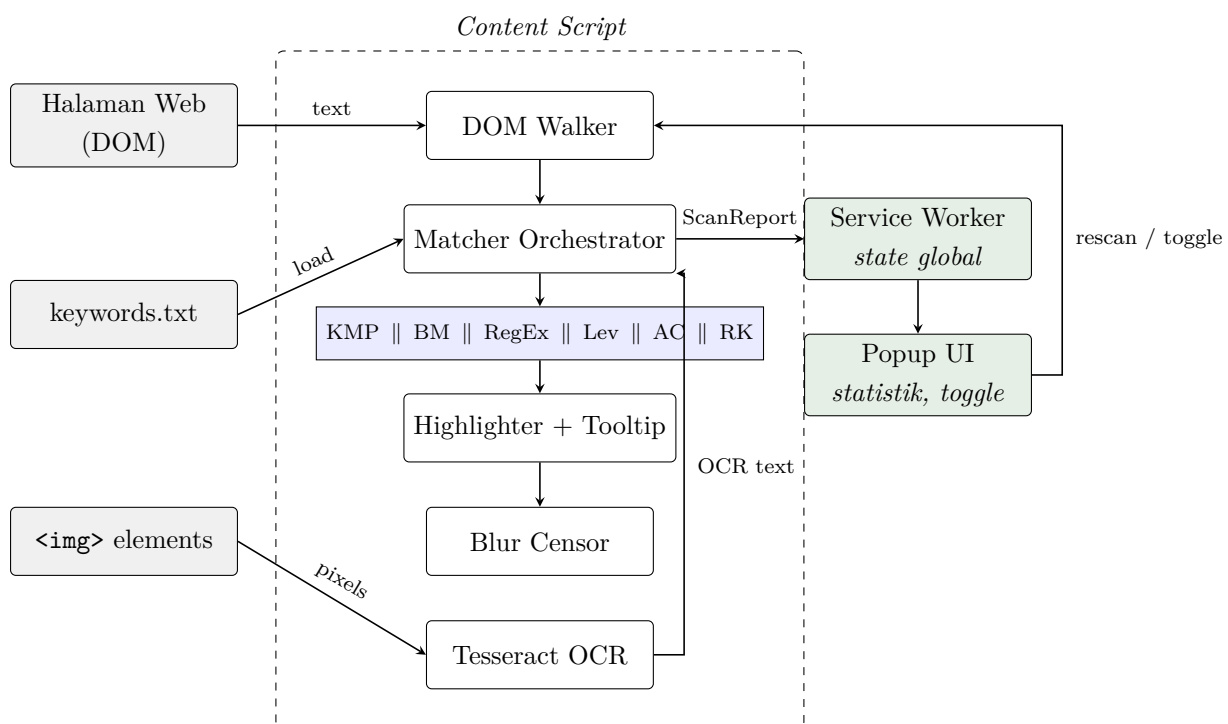


Figure 10: Arsitektur internal Judol Detector. *Content script* merangkum DOM walker, *matcher orchestrator* yang memanggil enam algoritma matching, modul *highlighter + tooltip*, *blur censor*, dan Tesseract OCR. *Service worker* memelihara *state* global, sementara popup menampilkan statistik beserta tombol kontrol.

3.5 Contoh Ilustrasi Kasus

Untuk memperlihatkan bagaimana pipeline bekerja secara konkret, kami ambil contoh teks dari halaman pencarian Google berikut.

```

1 Ajo89: Situs Slot Gacor Gampang Menang Link Rtp Hari Ini
2 Ajo89 adalah situs slot gacor dengan deposit rendah.
3 Slot Thailand Situs Slot Gacor Anti Rungkat - HOKI88

```

Misalkan daftar kata kunci kami berisi `slot gacor`, `slot`, `gacor`, `judi`, dan `HOKI88`, dan *threshold* fuzzy diatur 0.7. Hasil tiap tahap pipeline pada teks di atas tampak pada Tabel 4.

Table 4: *Trace* pipeline untuk teks contoh di atas. Angka comparison count bersifat ilustratif.

Tahap	Kemunculan yang tertangkap	Total	Catatan
Exact (KMP)	slot gacor (3×), slot (5×), gacor (3×)	11	Comparison count ≈ 312
Exact (BM)	sama dengan KMP	11	Comparison count ≈ 198 (lebih efisien karena pola panjang)
RegEx	Ajo89 (2×), HOKI88 (1×)	3	Tangkapan murni dari pola <code><kata><angka></code>
Fuzzy	HOKI88 $\rightarrow$ HOKI88	1	Similarity ≈ 0.92 , di atas <i>threshold</i>
Hasil akhir	Setelah <i>dedup</i> KMP/BM dan overlap RegEx/Fuzzy	14 unik	Highlighter membungkus 14 lokasi DOM

Di sini, tahap eksak menangkap kata kunci yang sudah dikenal, tahap RegEx menemukan Ajo89 yang belum pernah masuk daftar, dan tahap *fuzzy* menangkap penyamaran HOKI88. Walaupun HOKI88 juga tertangkap RegEx, label algoritmanya tetap dibawa terpisah, RegEx melaporkannya sebagai *discovery hit*, fuzzy melaporkannya sebagai *match* dari kata kunci HOKI88. Saat *toggle blur* aktif, seluruh elemen yang teridentifikasi akan di-blur sampai pengguna meng-hover atau mematikan toggle dari popup.

4 Implementasi dan Pengujian

4.1 Spesifikasi Teknis Program

Judul Detector dibangun di atas *Vite 8* dengan *TypeScript 6* (mode `strict` dengan `verbatimModuleSyntax`). Popup UI memakai *React 19*, sedangkan bundling sebagai ekstensi Chromium ditangani oleh plugin `@crxjs/vite-plugin`. Unit test untuk modul algoritma memakai *Vitest 4* pada lingkungan Node, sementara manajemen *dependency* memakai *pnpm* dengan lockfile yang ikut di-commit. Berkas konfigurasi ekstensi `public/manifest.json` mendeklarasikan permission dan komponen Manifest V3.

Lima kontrak data lintas modul dirumuskan pada `src/types/index.ts`, yang isinya tampak pada Listing 1. Kontrak ini sengaja dipatok di awal proyek supaya tiap orang bisa kerja paralel tanpa saling tunggu, dan supaya unit test algoritma bisa diuji secara terpisah dari DOM.

Listing 1: Kontrak data lintas modul (`src/types/index.ts`).

```
1 export type AlgorithmName =  
2   'KMP' | 'BoyerMoore' | 'RegEx' | 'Levenshtein' | 'AhoCorasick' | 'RabinKarp';  
3  
4 export interface MatchResult {  
5   keyword: string;  
6   matchedText: string;  
7   algorithm: AlgorithmName;  
8   startIndex: number;  
9   endIndex: number;  
10  comparisonCount: number;  
11  isFuzzy: boolean;  
12  similarity?: number;  
13 }  
14  
15 export interface AlgorithmStats {  
16   algorithm: AlgorithmName;  
17   executionTimeMs: number;  
18   matchCount: number;  
19   comparisonCount: number;  
20 }  
21  
22 export interface ScanReport {  
23   totalMatches: number;  
24   matchesByKeyword: Record<string, number>;  
25   stats: AlgorithmStats[];  
26   timestamp: number;  
27 }  
28  
29 export interface PatternMatcher {  
30   name: AlgorithmName;  
31   search(text: string, patterns: string[]): MatchResult[];  
32 }
```

Pembagian modul utama dan tanggung jawabnya diringkas pada Tabel 5. Modul dipecah

berdasarkan tanggung jawab, bukan berdasarkan komponen Manifest V3, supaya tiap algoritma bisa diuji terpisah dari DOM dan dari popup.

Table 5: Modul utama Judol Detector dan tanggung jawabnya.

Modul	Tanggung jawab
<code>types/index.ts</code>	Kontrak data lintas modul (<code>MatchResult</code> , <code>AlgorithmStats</code> , <code>ScanReport</code> , <code>PatternMatcher</code>).
<code>algorithms/kmp.ts</code>	Implementasi KMP <i>from scratch</i> dengan <i>border function</i> dan <i>comparison counting</i> .
<code>algorithms/boyerMoore.ts</code>	Implementasi Boyer-Moore dengan <i>last occurrence table</i> (heuristik <i>bad character</i>).
<code>algorithms/regex.ts</code>	Pola <code><kata><angka></code> dan <i>discovery layer</i> .
<code>algorithms/levenshtein.ts</code>	Weighted Levenshtein dengan tabel bobot visual-similarity dan threshold.
<code>algorithms/ahoCorasick.ts</code>	Implementasi Aho-Corasick (bonus).
<code>algorithms/rabinKarp.ts</code>	Implementasi Rabin-Karp dengan <i>rolling hash</i> (bonus).
<code>algorithms/ keywordLoader.ts</code>	Membaca <code>keywords.txt</code> dan mengubahnya jadi daftar pola.
<code>algorithms/ matcherOrchestrator.ts</code>	Menggabungkan pipeline tiga tahap, mendedup hits, mengumpulkan <i>stats</i> per algoritma.
<code>content/domWalker.ts</code>	Menelusuri DOM dan mengumpulkan <code>TextNode</code> .
<code>content/highlighter.ts</code>	Membungkus <i>match</i> dengan <code></code> , plus <code>clearHighlights()</code> .
<code>content/tooltip.ts</code>	Tooltip kustom yang muncul saat <i>hover</i> .
<code>content/content.ts</code>	<i>Entry point</i> content script, hook ke <code>DOMContentLoaded</code> dan <code>MutationObserver</code> .
<code>popup/</code>	UI popup berbasis React: statistik, grafik, tombol <i>rescan</i> dan <i>toggle blur</i> .
<code>background.ts</code>	<i>Service worker</i> Manifest V3: menyimpan <i>state</i> global antar tab.
<code>public/manifest.json</code>	Manifest ekstensi (nama, versi, permission, daftar komponen).

4.2 Tata Cara Penggunaan Ekstensi

Alur penggunaan ekstensi dari instalasi sampai melihat statistik tergambar pada Gambar 11. Karena ekstensi berjalan otomatis begitu halaman dimuat, pengguna sebetulnya tidak perlu interaksi tambahan untuk memulai pemindaian, namun masih bisa membuka popup kalau ingin melihat statistik atau mengganti pengaturan.

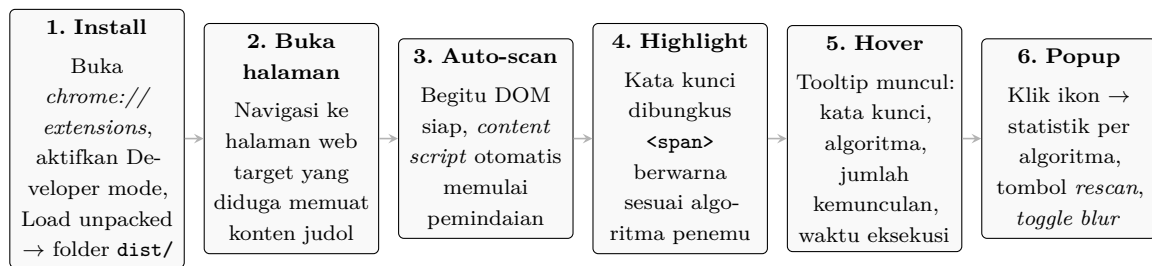


Figure 11: Tata cara penggunaan Judol Detector, dari instalasi sampai melihat statistik di popup. Pemindaian berjalan otomatis, interaksi pengguna baru diperlukan saat ingin melihat detail (hover) atau ringkasan (popup).

Tombol *rescan* di popup berguna untuk halaman yang kontennya berubah dinamis dan belum sempat ter-deteksi oleh `MutationObserver`. *Toggle blur* mengubah *state* global di *service worker* dan langsung berimbas ke semua tab yang sedang terbuka. Warna highlight per algoritma sengaja dibedakan supaya pengguna bisa melihat secara langsung kontribusi tiap algoritma terhadap deteksi.

4.3 Hasil Pengujian

Pengujian dilakukan dengan lima kasus yang memvariasikan jenis halaman dan jenis kemunculan kata kunci. Untuk tiap kasus, ekstensi dijalankan dengan *threshold fuzzy* 0.7 dan daftar kata kunci yang sama (`keywords.txt` versi yang di-commit). Browser yang digunakan adalah Google Chrome versi stabil pada platform yang konsisten antar pengujian.

4.3.1 Kasus 1: Hasil Pencarian Google dengan Kata Kunci Judol

Halaman: hasil pencarian Google untuk kata kunci “slot gacor”. Banyak situs judol muncul di SERP dengan nama berpola `<kata><angka>`, jadi kasus ini menguji tahap exact (KMP, BM) sekaligus tahap RegEx pada konten kepadatan tinggi.

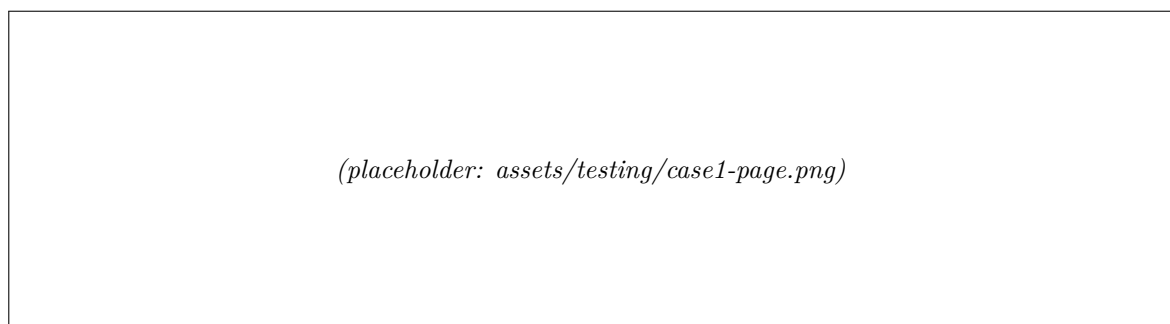


Figure 12: Kasus 1: hasil pencarian Google dengan highlight aktif. *Screenshot disiapkan: halaman SERP dengan beberapa judul situs judol yang sudah di-highlight, tooltip terlihat saat hover pada salah satu hit.*

(placeholder: assets/testing/case1-popup.png)

Figure 13: Kasus 1: popup ekstensi dengan statistik per algoritma. *Screenshot disiapkan: popup yang menampilkan total match, jumlah match dan comparison count per algoritma, dan grafik perbandingan waktu eksekusi.*

4.3.2 Kasus 2: Komentar Spam Judol pada Halaman Video

Halaman: kolom komentar video YouTube atau forum publik yang terkenal sering disisipi komentar judol. Kasus ini menguji DOM walker pada struktur yang dalam dan banyak `TextNode`, dengan kemunculan kata kunci tersebar dan diselingi karakter non-ASCII.

(placeholder: assets/testing/case2-page.png)

Figure 14: Kasus 2: kolom komentar dengan highlight pada beberapa komentar berisi kata kunci judol. *Screenshot: bagian komentar dengan minimal tiga highlight, tooltip aktif pada salah satunya.*

(placeholder: assets/testing/case2-popup.png)

Figure 15: Kasus 2: popup dengan statistik. *Screenshot: popup menampilkan distribusi kata kunci dan perbandingan waktu antar algoritma.*

4.3.3 Kasus 3: Halaman Situs Judol Langsung

Halaman: salah satu situs berpola `<kata><angka>` (misalnya halaman utama GACOR99 atau MAXWIN88) yang isinya didominasi konten judi. Kasus ini menguji kemampuan ekstensi pada

kepadatan match yang sangat tinggi, sekaligus mengukur dampak performa saat hampir setiap kalimat memicu match.

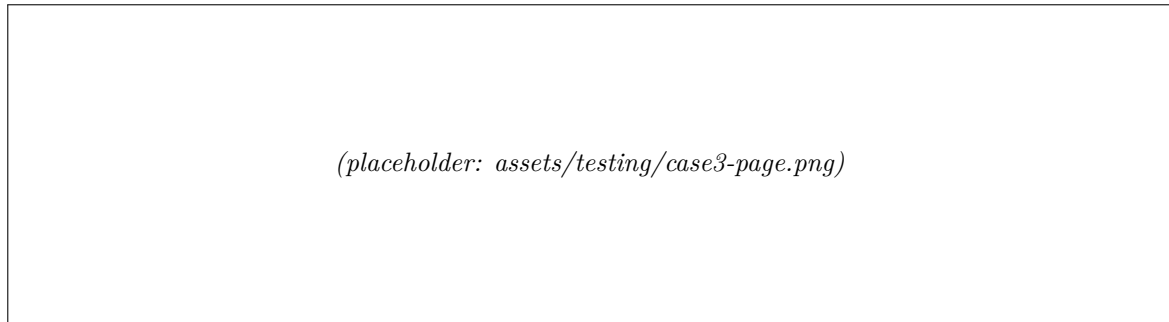


Figure 16: Kasus 3: situs judul dengan hampir seluruh teks ter-highlight. *Screenshot: tampilan halaman utama situs judul dengan highlight padat, sertakan juga area yang sudah di-blur kalau toggle blur aktif.*

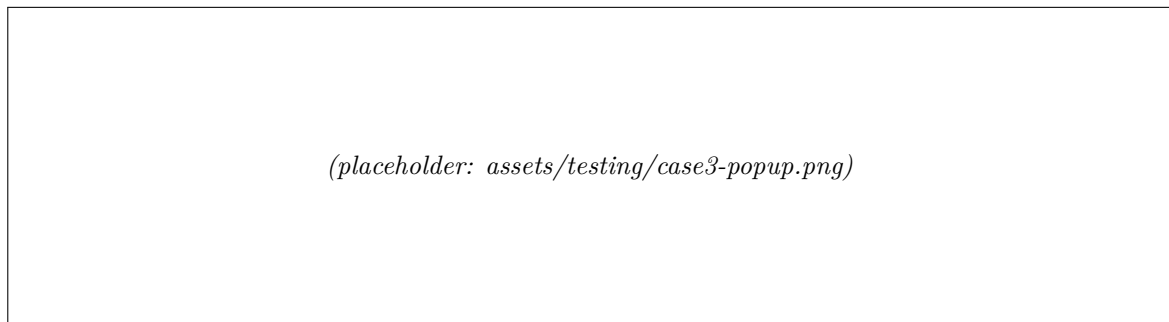


Figure 17: Kasus 3: popup dengan statistik kepadatan tinggi. *Screenshot: popup yang menampilkan total match jauh lebih tinggi dari kasus lainnya, perhatian utama pada comparison count.*

4.3.4 Kasus 4: Halaman dengan Penyamaran Karakter Visual

Halaman: halaman yang kontennya sengaja memakai substitusi karakter (H0KI88, Gacor999 dengan Unicode a yang berbeda, SLOT bukan SL0T). Kasus ini fokus menguji tahap fuzzy (Weighted Levenshtein) yang seharusnya tetap menangkap kemunculan walau exact dan RegEx gagal.

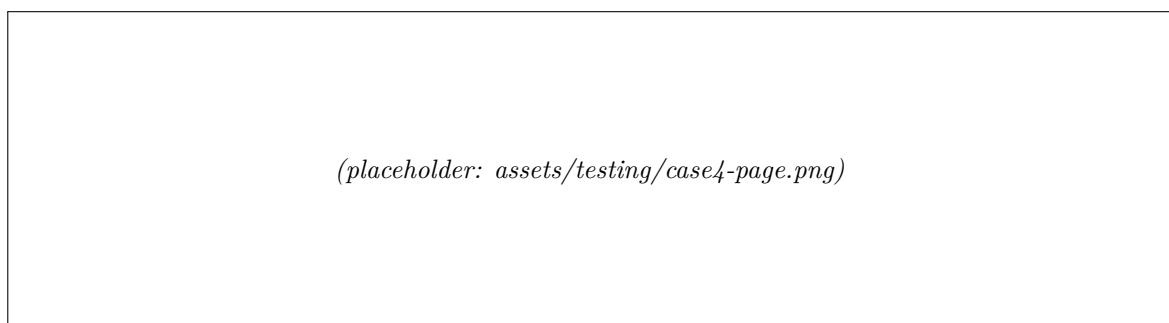


Figure 18: Kasus 4: halaman dengan penyamaran karakter, highlight muncul dari hasil fuzzy match. *Screenshot: tooltip aktif pada salah satu hit fuzzy, menampilkan algorithm: Levenshtein dan skor similarity.*

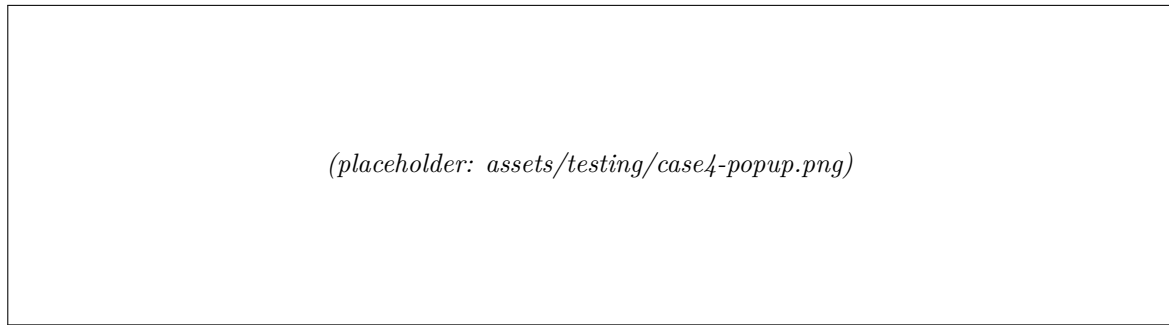


Figure 19: Kasus 4: popup memperlihatkan kontribusi tahap fuzzy. *Screenshot: popup yang menampilkan baris Levenshtein dengan jumlah match > 0 dan kolom KMP/BM yang lebih sedikit dibanding kasus lain.*

4.3.5 Kasus 5: Iklan Judol Berbentuk Gambar (OCR)

Halaman: halaman yang memuat iklan judol dalam bentuk gambar (misalnya banner statis yang tidak punya teks DOM). Kasus ini menguji fitur bonus OCR via Tesseract.js: gambar dibaca, dicocokkan ke daftar kata kunci, lalu di-blur kalau positif.

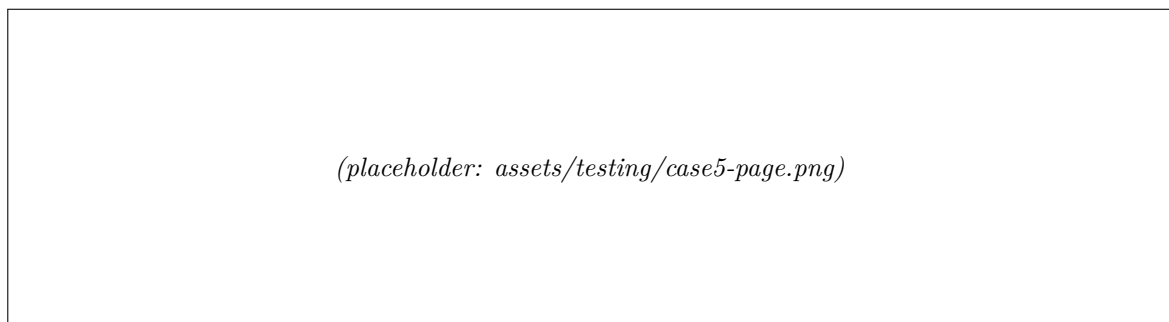


Figure 20: Kasus 5: gambar iklan judol yang sudah di-blur oleh ekstensi. *Screenshot: tampilan sebelum/sesudah blur (boleh dijadikan dua gambar berdampingan), plus tooltip yang menampilkan teks hasil OCR sebagai kata kunci.*

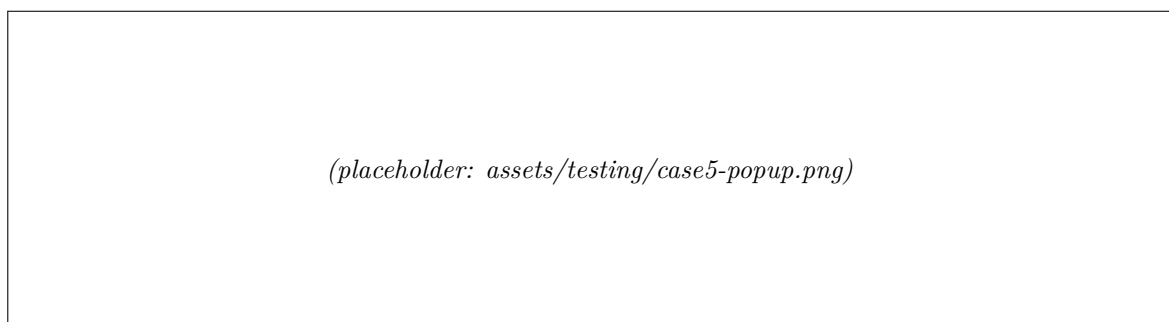


Figure 21: Kasus 5: popup memperlihatkan jumlah gambar yang dipindai dan jumlah yang positif. *Screenshot: popup dengan baris “OCR” atau “Image scan” yang menunjukkan kontribusinya.*

4.4 Analisis Perbandingan Performa

Tabel 6 merangkum metrik per algoritma untuk kelima kasus. Untuk tiap kasus, dicatat waktu eksekusi (`executionTimeMs`), jumlah *match*, dan jumlah perbandingan karakter (`comparisonCount`).

Karena KMP dan BM jalan paralel terhadap pola yang sama, jumlah *match* keduanya seharusnya identik, namun waktu dan *comparison count*-nya berbeda. Angka pada tabel diisi setelah pengujian dijalankan.

Table 6: Ringkasan performa per algoritma per kasus. Waktu dalam ms, comparison count dalam satuan ribu. *Diisi setelah pengujian dijalankan.*

Algoritma	Kasus 1	Kasus 2	Kasus 3	Kasus 4	Kasus 5
KMP (ms)	—	—	—	—	—
Boyer-Moore (ms)	—	—	—	—	—
RegEx (ms)	—	—	—	—	—
Levenshtein (ms)	—	—	—	—	—
Aho-Corasick (ms)	—	—	—	—	—
Rabin-Karp (ms)	—	—	—	—	—
KMP (comp.)	—	—	—	—	—
Boyer-Moore (comp.)	—	—	—	—	—
Aho-Corasick (comp.)	—	—	—	—	—
Rabin-Karp (comp.)	—	—	—	—	—
Total match	—	—	—	—	—
Match dari fuzzy	—	—	—	—	—

Hipotesis yang ingin dibuktikan dari data tersebut:

- Boyer-Moore akan unggul terhadap KMP dalam jumlah perbandingan karakter, terutama untuk kata kunci panjang seperti `slot gacor`, karena karakter akhirnya jarang sama dengan akhir teks dan pola bisa langsung melompat.
- Aho-Corasick akan unggul jauh untuk kasus dengan banyak kata kunci, karena teks cukup ditelusuri sekali untuk semua pola, sementara KMP dan BM harus menelusuri ulang per kata kunci.
- Rabin-Karp akan kompetitif untuk pola pendek, tapi mulai kalah saat sering terjadi tabrakan hash. Untuk pola panjang dan teks padat, kemungkinan ada banyak *spurious hit*.
- Weighted Levenshtein hanya jalan untuk kata kunci yang miss di tahap exact, jadi total waktunya bergantung pada seberapa banyak *miss list*-nya. Kasus 4 seharusnya jadi kontribusi terbesarnya.
- RegEx akan paling konsisten antar kasus karena pola tunggal dan engine native JavaScript, tapi tidak menggantikan KMP/BM karena tidak menyaring berdasarkan daftar kata kunci.

Analisis lanjutan, termasuk grafik perbandingan waktu eksekusi rata-rata per algoritma, akan ditambahkan setelah semua data hasil pengujian terkumpul.

5 Kesimpulan dan Saran

References

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.
- [2] “What is the best/worst time complexity of KMP algorithm?,” Stack Overflow, Sep. 2020. [Online]. Available: <https://stackoverflow.com/questions/63867161/what-is-the-best-worst-time-complexity-of-kmp-algorithm>
- [3] “Boyer-Moore Algorithm for Pattern Searching,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/dsa/boyer-moore-algorithm-for-pattern-searching/>
- [4] “Aho-Corasick algorithm,” Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Aho-Corasick_algorithm

Appendices

Statement

GitHub Repository

https://github.com/ethj0r/Tubes3_LeGamblers

Video Link

Spesification Checklist

Task Distribution